Curso: Estructuras de Datos
Periodo: I Cuatrimestre 2026
Prof. Alexander Jiménez Palacios

Simulación de Elevadores

1. Descripción General

El problema consiste en simular el funcionamiento de uno o varios elevadores dentro de un edificio de múltiples pisos. El sistema debe gestionar solicitudes de usuarios que desean desplazarse entre distintos pisos y determinar cómo los elevadores deben atender dichas solicitudes.

El objetivo es minimizar el tiempo de espera de los usuarios y optimizar la cantidad de movimientos realizados por los elevadores.

Cada solicitud incluye:

- Piso de origen
- Piso de destino
- Dirección del viaje (subir o bajar)

El sistema debe simular el movimiento del elevador y registrar el comportamiento del sistema.

2. Objetivo del Proyecto

Desarrollar un sistema que:

- Permita definir un edificio con múltiples pisos.
- Permita generar solicitudes de usuarios.
- Simule el movimiento de uno o más elevadores.
- Determine cómo se atienden las solicitudes.
- Compare diferentes estrategias de despacho.
- Analice la eficiencia del sistema.

3. Requerimientos Funcionales

El sistema debe permitir:

- Definir el número de pisos del edificio.
- Definir el número de elevadores.
- Registrar solicitudes de transporte entre pisos.
- Simular el movimiento del elevador paso a paso.
- Mostrar el estado actual del elevador.

Debe mostrar:

- Piso actual del elevador
- Dirección del movimiento
- Personas dentro del elevador
- Solicitudes pendientes

4. Requerimientos Técnicos Obligatorios

4.1 Estructuras de Datos

Debe utilizar al menos:

- Una estructura lineal (cola de solicitudes).
- Una estructura no lineal (grafo o árbol para modelar el edificio o las rutas).

Ejemplo de estructuras posibles:

- Cola de solicitudes de usuarios
- Lista de paradas programadas
- Tabla hash de usuarios activos

4.2 Algoritmos

Debe implementarse al menos:

1. Estrategia básica:
   - Atender solicitudes en orden de llegada.
2. Estrategia optimizada:

Por ejemplo:

- SCAN (similar a algoritmos de disco).
- Priorizar solicitudes en la dirección actual.
- Agrupar paradas cercanas.

4.3 Documentación

El README debe incluir:

- Número total de movimientos.
- Tiempo promedio de espera.
- Comparación entre estrategias.
- Complejidad de cada estrategia.

5. Ejemplo de Entrada

```
Número de pisos: 10
Elevadores: 1

Solicitudes:
Piso 2 -> Piso 8
Piso 5 -> Piso 1
Piso 7 -> Piso 9
```

6. Ejemplo de Salida Esperada

```
Estado del elevador:

Elevador en piso: 2
Dirección: Subiendo

Paradas programadas:
5, 7, 8, 9

Métricas:
Movimientos totales: 18
Tiempo promedio de espera: 3.4
```

7. Validaciones Obligatorias

- No permitir pisos fuera del rango.

- No permitir destino igual al origen.
- Manejar solicitudes simultáneas.
- Manejar edificio vacío.

8. Interfaz

Debe permitir:

- Ingresar solicitudes.
- Ejecutar simulación.
- Mostrar estado del sistema.
- Reiniciar simulación.

9. Aspectos de Evaluación Clave

Se evaluará especialmente:

- Correcto manejo de colas de solicitudes.
- Estrategia de despacho.
- Simulación clara del sistema.
- Análisis comparativo de algoritmos.

10. Consideraciones en la Defensa

Durante la defensa los estudiantes deben explicar:

- Cómo se organizan las solicitudes.
- Qué estrategia reduce movimientos.
- Cómo se mide eficiencia del sistema.

Recorrido del Caballo

1. Descripción General

El problema del Recorrido del Caballo consiste en determinar si un caballo de ajedrez puede recorrer todas las casillas de un tablero de tamaño $n \times n$, visitando cada casilla exactamente una vez, siguiendo las reglas de movimiento del ajedrez.

El caballo se mueve en forma de "L":

- Dos casillas en una dirección (horizontal o vertical).
- Una casilla en dirección perpendicular.

El objetivo es modelar y resolver este problema utilizando estructuras de datos adecuadas y analizar la eficiencia del algoritmo implementado.

2. Objetivo del Proyecto

Desarrollar un sistema que:

1. Permita ingresar el tamaño del tablero n.
2. Permita definir la posición inicial del caballo.
3. Determine si existe un recorrido completo.
4. Muestre el orden en que se visitan las casillas.
5. Compare al menos dos enfoques de resolución.
6. Analice la complejidad temporal y espacial.

3. Requerimientos Funcionales

El sistema debe permitir:

- Ingresar el tamaño del tablero (mínimo 5).
- Ingresar la posición inicial.
- Ejecutar el algoritmo.
- Mostrar:
    - El tablero con numeración del recorrido.
    - Si existe o no solución.

- Métricas de ejecución.

4. Requerimientos Técnicos Obligatorios

4.1 Estructuras de Datos

Debe utilizar al menos:

- Una estructura lineal (ej. lista o pila para gestionar movimientos).
- Una estructura no lineal (ej. modelado del tablero como grafo implícito o explícito).

No se permite resolver únicamente con arreglos simples sin justificación formal.

4.2 Algoritmos

Debe implementarse:

1. Una versión base utilizando backtracking.
2. Una versión optimizada o alternativa, por ejemplo:
    - Aplicación de heurística de menor grado (similar a Warnsdorff).
    - Ordenamiento de movimientos antes de explorar.
    - Estrategia alternativa de recorrido.

4.3 Documentación

El README debe incluir:

- Complejidad temporal teórica del algoritmo base.
- Complejidad espacial.
- Número de llamadas recursivas.
- Tiempo de ejecución.
- Comparación entre versiones.
- Discusión sobre escalabilidad al aumentar n.

5. Ejemplo de Entrada

Tamaño del tablero: 5
Posición inicial: (0,0)

6. Ejemplo de Salida Esperada

(El recorrido puede variar, pero debe ser válido)

Solución encontrada.

Tablero:

```
 1 16 11  6 21
10  5 20 15 12
19  2 17 22  7
 4  9 24 13 18
25 14  3  8 23
```

Métricas:
Llamadas recursivas: 12430
Tiempo de ejecución: 0.18 segundos
Versión heurística: 312 llamadas

7. Validaciones Obligatorias

- Verificar que la posición inicial sea válida.
- Verificar que n ≥ 5.
- Manejar caso donde no exista solución.
- No permitir movimientos fuera del tablero.

8. Interfaz

Debe incluir una interfaz en consola que permita:

- Ejecutar ambas versiones.
- Comparar resultados.
- Repetir pruebas con distintos valores de n.

9. Aspectos de Evaluación Clave

Se evaluará especialmente:

- Correcta implementación de backtracking.
- Justificación de estructuras.
- Comparación clara entre enfoques.

- Calidad del análisis de complejidad.
- Capacidad de explicar el árbol de exploración en defensa oral.

10. Consideraciones en la Defensa

Durante la defensa, los estudiantes deben poder explicar:

- Por qué el problema es exponencial.
- Cómo funciona el retroceso.
- Qué reduce el factor de ramificación.
- Qué sucede al aumentar el tamaño del tablero.
- Diferencias entre modelar como matriz o como grafo.

Sistema de Parqueo Inteligente

1. Descripción General

El problema consiste en diseñar e implementar un sistema que simule la operación de un **parqueo inteligente** con capacidad limitada. El sistema debe registrar la entrada y salida de vehículos, asignar espacios disponibles según una estrategia definida y gestionar una lista de espera cuando el parqueo se encuentre lleno.

El sistema debe permitir consultar el estado del parqueo en cualquier momento y generar métricas que permitan analizar el desempeño del proceso de asignación y atención (por ejemplo: ocupación, tiempos de espera, rotación de espacios, ingresos).

2. Objetivo del Proyecto

Desarrollar un sistema que:

- Permita definir la configuración del parqueo (cantidad de espacios, tipo de espacios).
- Registre la entrada y salida de vehículos.
- Asigne automáticamente espacios disponibles bajo una estrategia definida.
- Mantenga una lista de espera cuando no existan espacios disponibles.
- Permita consultar el estado del parqueo (ocupación, lista de espera, historial).
- Genere métricas y reportes para análisis de eficiencia.
- Compare al menos dos estrategias de asignación de espacios (cuando aplique).

3. Requerimientos Funcionales

El sistema debe permitir, como mínimo:

3.1 Configuración

- Definir la capacidad del parqueo (cantidad total de espacios).
- (Opcional) Definir tipos de espacios:
    - Normal
    - Preferencial (discapacidad/embarazadas)
    - Moto

- Eléctrico

3.2 Registro de Entrada

Para cada vehículo se debe ingresar:

- Placa (única)
- Tipo de vehículo (carro/moto) (mínimo)
- Hora/tiempo de llegada (si es simulación por ticks o eventos)

El sistema debe:

- Asignar un espacio si existe disponibilidad.
- Si el parqueo está lleno, colocar el vehículo en lista de espera.

3.3 Registro de Salida

Al registrar la salida:

- Se libera el espacio del vehículo.
- Se calcula el tiempo de permanencia.
- Se calcula el cobro (si aplica).
- Si hay lista de espera, se asigna el espacio recién liberado al primer vehículo en espera (o según estrategia).

3.4 Consultas y Reportes

El sistema debe mostrar:

- Estado actual del parqueo (ocupado/libre por espacio).
- Lista de espera.
- Búsqueda de un vehículo por placa y su ubicación.
- Reporte de métricas al finalizar (o bajo demanda).

4. Requerimientos Técnicos Obligatorios

4.1 Estructuras de Datos

Debe utilizar al menos:

Una estructura lineal

Ejemplos aceptables:

- Cola para lista de espera (FIFO)
- Cola de prioridad para lista de espera (por tipo de vehículo o antigüedad)
- Lista para historial de entradas/salidas

Una estructura no lineal

Ejemplos aceptables:

- Tabla hash para localizar vehículo por placa rápidamente
- Árbol (opcional) para reportes ordenados (por tiempo, cobro, etc.)

No se permite resolver todo con arreglos simples sin justificar formalmente el impacto en eficiencia.

4.2 Algoritmos

Debe implementarse al menos:

- Algoritmo de asignación de espacio (estrategia).
- Búsqueda eficiente por placa.
- Actualización de lista de espera.
- (Opcional pero recomendado) ordenamientos para reportes:
    - Top 5 espacios más usados
    - Top 5 vehículos con mayor permanencia
    - Horas pico (mayor ocupación)

4.3 Comparación de Enfoques (cuando aplique)

Debe compararse al menos una de estas variantes:

- **Estrategia A (Base):** asignar el primer espacio libre (secuencial).
- **Estrategia B (Mejorada):** asignación por cercanía a la salida, o por tipo de espacio (motos, preferenciales), o por prioridad.

La comparación debe incluir métricas (tiempo de espera, rotación, etc.).

4.4 Documentación (README)

El README.md debe incluir:

- Complejidad temporal y espacial de las operaciones principales:
  - Registrar entrada
  - Registrar salida
  - Buscar vehículo por placa
  - Asignar espacio
- Métricas experimentales:
  - Ocupación promedio
  - Tiempo promedio de espera (si hubo lista)
  - Tiempo máximo de espera
  - Rotación de espacios (cuántas veces se utilizó cada espacio)
- Comparación de estrategias (si se implementan dos).

5. Ejemplo de Entrada

Configuración:

- Espacios: 3

Eventos:

```
ENTRA ABC123 carro
ENTRA DEF456 carro
ENTRA GHI789 moto
ENTRA JKL111 carro
SALE  DEF456
ENTRA MNO222 carro
```

6. Ejemplo de Salida Esperada

```
ENTRA ABC123 -> asignado a espacio #1
ENTRA DEF456 -> asignado a espacio #2
ENTRA GHI789 -> asignado a espacio #3
ENTRA JKL111 -> parqueo lleno, enviado a lista de espera

SALE DEF456 -> libera espacio #2
```

```
ASIGNACIÓN AUTOMÁTICA -> JKL111 asignado a espacio #2

ENTRA MNO222 -> parqueo lleno, enviado a lista de espera

Estado actual:
#1 ABC123
#2 JKL111
#3 GHI789

Lista de espera:
MNO222
```

Métricas:
Ocupación promedio: 92%
Tiempo promedio de espera: 2.3
Espacio más usado: #2 (3 usos)

(Los valores pueden variar según la simulación, pero deben ser coherentes.)

7. Validaciones Obligatorias

- No permitir placas duplicadas dentro del parqueo o en espera.
- No permitir salida de una placa inexistente.
- Manejar parqueo vacío.
- Validar el tipo de vehículo y capacidad.
- Si existen espacios por tipo (moto/preferencial), validar asignación correcta.

8. Interfaz (Consola)

Debe incluir un menú que permita:

1. Configurar parqueo.
2. Registrar entrada.
3. Registrar salida.
4. Buscar vehículo por placa.
5. Mostrar estado del parqueo.
6. Mostrar lista de espera.

7. Generar reporte de métricas.
8. Salir.

9. Aspectos de Evaluación Clave

Se evaluará especialmente:

- Correcta asignación y liberación de espacios.
- Uso real y justificado de tabla hash para búsquedas.
- Manejo correcto de cola/lista de espera.
- Calidad del diseño modular (clases, separación).
- Métricas claras y comparables.
- Capacidad de defensa técnica.

10. Consideraciones en la Defensa

Durante la defensa, los estudiantes deben poder explicar:

- Por qué se usó tabla hash (y no lista) para buscar por placa.
- Qué estructura se usó para lista de espera y por qué.
- Cómo se decide la asignación de espacios.
- Qué métricas recolectaron y qué conclusiones obtuvieron.
- Complejidad temporal y espacial de operaciones principales.

N-Reinas

1. Descripción General

El problema de las **N-Reinas** consiste en ubicar **N reinas** en un tablero de ajedrez de tamaño $N \times N$de forma que **ninguna reina ataque a otra**.

Una reina ataca a otra si comparten:

- **La misma fila**
- **La misma columna**
- **La misma diagonal** (principal o secundaria)

Este problema es un clásico para aplicar **backtracking**, **poda** y análisis de eficiencia, ya que el espacio de búsqueda crece muy rápido conforme aumenta N.

2. Objetivo del Proyecto

Desarrollar un sistema que:

- Permita ingresar un valor N (tamaño del tablero).
- Encuentre **todas las soluciones posibles** para ese N(o un número máximo configurable de soluciones si Nes alto).
- Muestre cada solución en formato tablero o como lista de coordenadas.
- Compare al menos **dos enfoques/variantes** de resolución.
- Analice la complejidad temporal y espacial, con métricas reales de ejecución.

3. Requerimientos Funcionales

El sistema debe permitir:

1. **Ingreso de parámetros**
    - Ingresar N(mínimo 4).
    - (Opcional) Límite máximo de soluciones a imprimir (para Ngrandes).
2. **Ejecución del algoritmo**
    - Ejecutar la búsqueda de soluciones.
    - Mostrar resultados.

3. **Salida / Resultados**
    - Número total de soluciones encontradas.
    - Representación de al menos:
        - Tablero con "Q" para reinas y "." para vacíos
        - Lista de coordenadas $(fila, columna)$por solución.
    - Métricas de ejecución (tiempo, intentos/backtracks).

4. Requerimientos Técnicos Obligatorios

4.1 Estructuras de Datos

Debe utilizar al menos:

Una estructura lineal

Ejemplos aceptables:

- Lista para almacenar soluciones encontradas
- Pila implícita (recursión) para backtracking
- Lista para guardar posiciones por fila

Una estructura no lineal

Ejemplos aceptables:

- **Tabla hash** para registrar estados visitados o configuraciones parciales (opcional, para variante)
- **Árbol implícito** de búsqueda (se debe explicar como modelo conceptual de la exploración)

Nota: El tablero puede representarse con matriz o con un arreglo/estructura de posiciones, pero **debe justificarse** formalmente la representación y su impacto en eficiencia.

4.2 Algoritmos (obligatorio comparar variantes)

Debe implementarse:

Variante A – Backtracking básico (base)

- Colocar una reina por fila.

- Probar columnas disponibles.
- Verificar ataques en columnas y diagonales.
- Retroceder si no hay opciones válidas.

Variante B – Backtracking optimizado (poda)

Debe incluir al menos una optimización, por ejemplo:

- Uso de estructuras auxiliares para ataques:
  - columnas ocupadas
  - diagonales principales
  - diagonales secundarias
- Reducción de validaciones repetidas.
- (Opcional) Simetría: contar soluciones reflejadas/reducir exploración (para N pares).

4.3 Documentación (README)

El README.md debe incluir:

- Complejidad temporal y espacial (teórica) de ambas variantes.
- Número de:
  - intentos (posiciones probadas)
  - retrocesos (backtracks)
  - soluciones encontradas
- Tiempo de ejecución.
- Comparación clara entre variantes:
  - ¿Cuál fue más rápida?
  - ¿Por qué?
- Discusión de escalabilidad:
  - qué ocurre al aumentar N

5. Ejemplo de Entrada

N = 4

(Opcional)

Máximo de soluciones a imprimir = 2

6. Ejemplo de Salida Esperada

Soluciones encontradas: 2

Solución 1:
. Q . .
. . . Q
Q . . .
. . Q .

Solución 2:
. . Q .
Q . . .
. . . Q
. Q . .

Métricas:
Variante A (base):
- Intentos: 64
- Backtracks: 44
- Tiempo: 0.012s

Variante B (optimizada):
- Intentos: 28
- Backtracks: 12
- Tiempo: 0.004s

(Los valores pueden variar, pero deben ser coherentes y justificar la mejora.)

7. Validaciones Obligatorias

- Validar que $N \geq 4$.
- Manejar correctamente:

- Caso sin soluciones (por ejemplo, si el usuario intenta $N = 2$o $N = 3$, debe indicar que no hay).

- Evitar imprimir miles de tableros sin control (para Ngrandes):
  - Permitir limitar impresión de soluciones, pero **mantener conteo total** (si el enfoque lo permite).

8. Interfaz (Consola)

Debe incluir un menú que permita:

1. Ingresar N
2. Ejecutar Variante A (base)
3. Ejecutar Variante B (optimizada)
4. Mostrar soluciones (con límite configurable)
5. Mostrar métricas comparativas
6. Salir

9. Aspectos de Evaluación Clave

Se evaluará especialmente:

- Correcta implementación de backtracking.
- Correcta validación de ataques (columna/diagonales).
- Implementación real de poda/optimización en la segunda variante.
- Justificación técnica de estructuras elegidas.
- Métricas claras y comparables.
- Capacidad de explicar el árbol de búsqueda en la defensa oral.

10. Consideraciones en la Defensa

Durante la defensa, los estudiantes deben poder explicar:

- Por qué el problema crece de forma exponencial.
- Cómo funciona el retroceso (backtracking) en este caso.
- Qué cambia entre la variante base y la optimizada.

- Qué estructuras reducen el costo de validación.
- Cómo se interpreta el problema como un árbol de exploración.

Laberintos (Generar y Resolver)

1. Descripción General

El proyecto consiste en construir un sistema capaz de:

1. **Generar automáticamente** un laberinto de tamaño $n \times m$(con paredes y caminos).
2. **Resolver** el laberinto encontrando un camino desde una entrada hasta una salida.

El laberinto puede modelarse como:

- Una **matriz** de celdas (pared/camino)
- Un **grafo implícito** donde cada celda transitable es un nodo y sus vecinos son aristas.

Este proyecto permite aplicar estructuras lineales y no lineales de forma natural, además de comparar estrategias de generación y búsqueda.

2. Objetivo del Proyecto

Desarrollar un sistema que:

- Permita definir las dimensiones del laberinto (n filas, $m$columnas).
- Genere un laberinto válido y transitable.
- Permita definir (o asignar automáticamente) una **entrada** y una **salida**.
- Encuentre y muestre una ruta solución.
- Compare al menos dos enfoques (ya sea de generación o de resolución).
- Analice complejidad temporal y espacial, y reporte métricas reales de ejecución.

3. Requerimientos Funcionales

El sistema debe permitir:

3.1 Configuración del laberinto

- Ingresar tamaño $n \times m$(mínimo recomendado: 10×10).
- Definir entrada y salida (o auto-asignarlas en bordes).
- (Opcional) Usar una semilla para reproducibilidad del laberinto.

3.2 Generación

- Generar un laberinto donde exista al menos un camino entre entrada y salida.
- Mostrar el laberinto generado en consola, por ejemplo:
  - # = pared
  - . = camino
  - E = entrada
  - S = salida

3.3 Resolución

- Resolver el laberinto y mostrar:
  - Ruta encontrada (marcada con * o lista de coordenadas).
  - Si no hay ruta (en caso de laberintos no garantizados), indicar “sin solución”.

3.4 Reportes y métricas

- Mostrar métricas del proceso de generación y de resolución.

4. Requerimientos Técnicos Obligatorios

4.1 Estructuras de Datos

Debe utilizar al menos:

Una estructura lineal

Ejemplos aceptables:

- **Pila** para generación (DFS iterativo) o para reconstruir ruta
- **Cola** para resolución (BFS)
- Lista para almacenar ruta final

Una estructura no lineal

Ejemplos aceptables:

- **Grafo** (explícito o implícito) para representar celdas y conexiones
- **Tabla hash** para marcar visitados (coordenadas → visitado) en una variante
- Árbol implícito de búsqueda (explicar el modelo)

No se permite resolver todo solo con arreglos simples sin justificación. Si usan matriz para el laberinto, deben justificar por qué y complementar con estructuras lineales/no lineales para el algoritmo.

4.2 Algoritmos (obligatorio comparar variantes)

Debe implementarse y compararse al menos una de estas opciones:

Opción A (Recomendada): Comparar algoritmos de resolución

- **BFS** (encuentra camino más corto en número de pasos)
- **DFS** (no garantiza el más corto, pero es más simple y usa pila)

Opción B: Comparar algoritmos de generación

- Generación con DFS "carving" (laberinto perfecto)
- Generación aleatoria simple + validación de conectividad (y regenerar si no hay salida)

(Se permite hacer comparación tanto en generación como en resolución, pero mínimo una comparación debe existir.)

4.3 Documentación (README)

El README.md debe incluir:

- Complejidad temporal y espacial (Big-O) de:
  - Generación
  - Resolución BFS y/o DFS
- Métricas reales:
  - Tiempo de generación
  - Tiempo de resolución
  - Nodos/celdas exploradas
  - Longitud de ruta (pasos)
- Comparación entre variantes:
  - BFS vs DFS (o generación A vs B)
- Discusión de escalabilidad:

- Qué sucede al aumentar n y m

5. Ejemplo de Entrada

Filas = 10
Columnas = 15
Entrada = (0,1)
Salida = (9,13)
Semilla = 123 (opcional)

6. Ejemplo de Salida Esperada

(El laberinto puede variar, pero debe ser válido y la ruta debe conectar E con S.)

Laberinto generado:

```
###############
#E..#.....#...#
#.#.#.###.#.#.#
#.#...#...#.#.#
#.#####.###.#.#
#.....#.....#.#
###.#.#####.#.#
#...#.....#...#
#.#######.###.#
#.......#...S##
###############
```

Ruta encontrada (BFS):

```
###############
#E..#.....#...#
#*#.#.###.#.#.#
#*#...#...#.#.#
#*#####.###.#.#
#*****#.....#.#
###.#*#####.#.#
#...#*****#...#
#.#######*###.#
#.......#***S##
###############
```

Métricas:
BFS -> Exploradas: 92 celdas | Ruta: 18 pasos | Tiempo: 0.006s
DFS -> Exploradas: 141 celdas | Ruta: 33 pasos | Tiempo: 0.004s

(Los números pueden variar, pero deben ser coherentes: BFS no siempre es más rápido, pero sí debe dar ruta más corta en pasos.)

7. Validaciones Obligatorias

- Validar dimensiones mínimas.
- Validar que entrada y salida estén dentro del tablero y sean celdas transitables.
- Evitar laberintos inválidos (sin conexión) si el generador no lo garantiza:
    - Si el método puede producir laberintos sin solución, se debe manejar el caso:
        - Regenerar.
        - Reportar "sin solución".

8. Interfaz (Consola)

Debe incluir un menú que permita:

1. Definir tamaño $n \times m$
2. Generar laberinto (con semilla opcional)
3. Mostrar laberinto
4. Resolver con BFS
5. Resolver con DFS
6. Comparar métricas
7. Reiniciar / generar otro laberinto
8. Salir

9. Aspectos de Evaluación Clave

Se evaluará especialmente:

- Calidad y validez del algoritmo de **generación**.
- Correcta implementación de BFS y DFS.

- Manejo adecuado de visitados y reconstrucción de ruta.
- Comparación cuantitativa con métricas.
- Justificación de estructuras usadas.
- Diseño modular (clases / separación de responsabilidades).

10. Consideraciones en la Defensa

Durante la defensa, los estudiantes deben poder explicar:

- Cómo representan el laberinto (matriz vs grafo).
- Diferencia conceptual entre BFS y DFS.
- Por qué BFS encuentra el camino más corto (en pasos).
- Qué estructuras usan para:
    - Generar (pila/DFS)
    - Resolver (cola/BFS)
- Complejidad teórica y resultados medidos.

8-Puzzle / 15-Puzzle

1. Descripción General

El **8-Puzzle** (3×3) y el **15-Puzzle** (4×4) son rompecabezas de fichas deslizantes. El tablero contiene números y un espacio vacío (representado comúnmente como 0). En cada movimiento, el espacio vacío se intercambia con una ficha adyacente (arriba/abajo/izquierda/derecha).

El objetivo es encontrar una secuencia de movimientos que transforme un **estado inicial** en un **estado objetivo**, idealmente con el **mínimo número de movimientos**.

Este proyecto se presta naturalmente para modelar el problema como un **grafo de estados** y aplicar algoritmos de búsqueda (BFS/A*), usando estructuras como colas, colas de prioridad y tablas hash.

2. Objetivo del Proyecto

Desarrollar un sistema que:

- Permita seleccionar el tipo de puzzle: **8-Puzzle (3×3)** o **15-Puzzle (4×4)**.
- Permita ingresar un estado inicial (manual o aleatorio con semilla).
- Permita definir o usar un estado objetivo estándar.
- Encuentre una solución (si existe) y muestre:
  - Secuencia de movimientos
  - Número de movimientos
  - Estados intermedios (opcional, con límite)
- Compare al menos **dos enfoques de búsqueda** cuando sea aplicable.
- Analice complejidad temporal y espacial (teórica y empírica).

3. Requerimientos Funcionales

El sistema debe permitir:

3.1 Configuración

- Elegir modo:
  - 8-Puzzle (3×3)

  - 15-Puzzle (4×4)

- Ingresar el estado inicial:
  - Manual (matriz)
  - Aleatorio con semilla (para reproducibilidad)

3.2 Ejecución

- Ejecutar algoritmo de resolución.
- Mostrar resultados.

3.3 Resultados

Debe mostrar:

- Si existe solución o no.
- Número de movimientos.
- Secuencia de movimientos (por ejemplo: U, D, L, R).
- Métricas:
  - Nodos explorados
  - Tamaño máximo de frontera (cola)
  - Tiempo de ejecución

4. Requerimientos Técnicos Obligatorios

4.1 Estructuras de Datos

Debe utilizar al menos:

Una estructura lineal

- **Cola** para BFS
- Lista para reconstruir el camino solución (o pila)
- Lista/array para representar el estado (pero debe justificarse)

Una estructura no lineal

- **Tabla hash / set** para estados visitados (obligatorio)

- Grafo implícito de estados (modelo conceptual; puede ser explícito si desean)

No se permite buscar sin control de estados repetidos. Debe existir una estructura que evite reexplorar estados (hash).

4.2 Algoritmos (obligatorio comparar variantes)

Debe implementarse:

Variante A – BFS (Búsqueda en Anchura)

- Encuentra la solución con **menor número de movimientos**.
- Puede consumir mucha memoria, especialmente para 15-Puzzle.

Variante B – A* (A estrella) o búsqueda informada

- Usa una heurística para priorizar estados más prometedores.
- Requiere **cola de prioridad**.

Heurísticas permitidas (al menos una):

- Distancia Manhattan (recomendada)
- Número de fichas fuera de lugar

Si el grupo decide hacer solo 8-Puzzle, BFS y A* son ambos viables. Si hacen 15-Puzzle, BFS puede ser inviable en casos difíciles, por lo que A* es altamente recomendado.

4.3 Documentación (README)

El README.md debe incluir:

- Complejidad temporal y espacial (Big-O) esperada de BFS y A*.
- Explicación de la representación del estado y el hashing.
- Métricas reales:
    - nodos explorados
    - tamaño máximo de frontera
    - tiempo
- Comparación BFS vs A* en al menos 2 escenarios:

- Un caso “fácil”
- Un caso “medio” (más mezclado)

- Discusión de escalabilidad:
  - por qué 15-Puzzle crece muchísimo más que 8-Puzzle

5. Ejemplo de Entrada

5.1 8-Puzzle (3×3)

Estado inicial:

```
1 2 3
4 0 6
7 5 8
```

Estado objetivo:

```
1 2 3
4 5 6
7 8 0
```

6. Ejemplo de Salida Esperada

Solución encontrada.

Movimientos mínimos (BFS): 2
Secuencia: D, R

Estados:

```
1 2 3       1 2 3       1 2 3
4 0 6  ->   4 5 6  ->   4 5 6
7 5 8       7 0 8       7 8 0
```

Métricas:
BFS:
- Nodos explorados: 41
- Frontera máxima: 22
- Tiempo: 0.005s

A* (Manhattan):

- Nodos explorados: 9
- Frontera máxima: 6
- Tiempo: 0.001s

(Los números pueden variar, pero deben ser coherentes: A debería explorar menos nodos que BFS en la mayoría de casos.)*

7. Validaciones Obligatorias

- Validar que el estado inicial contenga todos los valores requeridos:
  - 0..8 para 8-Puzzle
  - 0..15 para 15-Puzzle
- Validar que no existan repetidos ni faltantes.
- Validar que el estado objetivo también sea válido.
- **Validación de solvencia (obligatoria):**
  - Si el estado inicial no tiene solución respecto al objetivo, el sistema debe detectarlo y reportarlo.
  - (No necesitan demostrar la teoría completa, pero sí implementar una verificación correcta para 3×3 y/o 4×4 según el modo elegido.)

8. Interfaz (Consola)

Debe incluir un menú que permita:

1. Elegir 8-Puzzle o 15-Puzzle
2. Ingresar estado inicial (manual o aleatorio con semilla)
3. Definir estado objetivo (opcional) o usar el estándar
4. Resolver con BFS
5. Resolver con A* (heurística seleccionable)
6. Mostrar métricas comparativas
7. Reiniciar / generar nuevo caso
8. Salir

9. Aspectos de Evaluación Clave

Se evaluará especialmente:

- Representación correcta del estado y generación de vecinos.
- Uso correcto de tabla hash para estados visitados.
- Reconstrucción correcta del camino solución.
- Implementación correcta de BFS y A* (cola vs cola de prioridad).
- Validación de solvencia.
- Métricas claras y comparación real.

10. Consideraciones en la Defensa

Durante la defensa, los estudiantes deben poder explicar:

- Cómo modelan el puzzle como un grafo de estados.
- Por qué BFS garantiza el mínimo número de movimientos.
- Qué es una heurística y por qué A* suele explorar menos.
- Cómo evitan ciclos/estados repetidos (hashing).
- Complejidad espacial: por qué BFS consume tanta memoria.
- Diferencias de escalabilidad entre 8-Puzzle y 15-Puzzle.

Sudoku Solver

1. Descripción General

El proyecto consiste en desarrollar un sistema capaz de **resolver automáticamente** un Sudoku estándar de **9×9**, respetando las reglas del juego:

- Cada **fila** debe contener los números del 1 al 9 sin repetir.
- Cada **columna** debe contener los números del 1 al 9 sin repetir.
- Cada **subcuadrante 3×3** debe contener los números del 1 al 9 sin repetir.

Un Sudoku se entrega como una cuadrícula parcialmente llena; las casillas vacías se representan típicamente con **0** o con ..

Este problema es ideal para aplicar **backtracking**, **validaciones eficientes**, y para comparar una versión básica vs una optimizada usando estructuras auxiliares.

2. Objetivo del Proyecto

Desarrollar un sistema que:

- Permita ingresar un Sudoku (9×9) con casillas vacías.
- Valide que el tablero inicial sea consistente (sin violar reglas).
- Resuelva el Sudoku utilizando backtracking.
- Muestre la solución final (si existe).
- Compare al menos dos enfoques o variantes de resolución.
- Analice complejidad temporal y espacial con métricas reales.

3. Requerimientos Funcionales

El sistema debe permitir:

3.1 Ingreso del tablero

- Ingresar el tablero manualmente (fila por fila).
- Cargarlo desde texto pegado en consola (opcional).
- Representar vacíos con 0.

3.2 Validación inicial

Antes de resolver, el sistema debe:

- Verificar que no existan conflictos en filas, columnas o subcuadrantes.
- Reportar error si el tablero inicial es inválido.

3.3 Resolución

- Ejecutar el algoritmo.
- Indicar si:
    - Hay solución única o al menos una solución (no es obligatorio detectar unicidad).
    - No existe solución.

3.4 Resultados

Debe mostrar:

- Tablero resuelto.
- Métricas de ejecución (tiempo, intentos, retrocesos).
- Comparación entre variantes.

4. Requerimientos Técnicos Obligatorios

4.1 Estructuras de Datos

Debe utilizar al menos:

Una estructura lineal

- Pila implícita (recursión) para backtracking
- Lista de posiciones vacías (recomendado) para ordenar el proceso
- (Opcional) pila explícita para resolver iterativamente

Una estructura no lineal

- Tabla hash o estructuras de control tipo “set” para validar rápidamente (permitido)
- Alternativamente, estructuras auxiliares booleanas por fila/columna/caja (también válido, pero deben justificarlo)

Importante: No se permite que toda la validación sea “recorrer todo el tablero” en cada intento sin justificar el impacto. Debe existir una variante optimizada de validación.

4.2 Algoritmos (obligatorio comparar variantes)

Debe implementarse:

Variante A – Backtracking básico

- Buscar la siguiente casilla vacía.
- Probar números del 1 al 9.
- Validar si es permitido.
- Repetir recursivamente.
- Retroceder si no hay opción válida.

Variante B – Backtracking optimizado (poda / heurística)

Debe incluir al menos una optimización, por ejemplo:

- **Validación O(1)** con estructuras auxiliares:
  - fila[9][10], col[9][10], box[9][10] (booleanos) o equivalente con hash/set.
- **Selección inteligente de casilla**:
  - Elegir primero la casilla con menos candidatos (MRV).
  - Mantener lista de vacíos y reordenar por número de opciones.
- (Opcional) Forward checking (actualizar candidatos).

4.3 Documentación (README)

El README.md debe incluir:

- Complejidad temporal y espacial (Big-O) aproximada del backtracking.
- Explicación de la diferencia entre variante básica y optimizada.
- Métricas reales para ambas variantes:
  - Intentos (números probados)
  - Retrocesos (backtracks)
  - Tiempo de ejecución

- Comparación en al menos 2 tableros:
  - Uno “fácil”
  - Uno “medio/difícil”
- Discusión sobre escalabilidad:
  - Por qué algunos Sudokus requieren mucho más backtracking.

5. Ejemplo de Entrada

Tablero (0 = vacío):

```
5 3 0 0 7 0 0 0 0
6 0 0 1 9 5 0 0 0
0 9 8 0 0 0 0 6 0
8 0 0 0 6 0 0 0 3
4 0 0 8 0 3 0 0 1
7 0 0 0 2 0 0 0 6
0 6 0 0 0 0 2 8 0
0 0 0 4 1 9 0 0 5
0 0 0 0 8 0 0 7 9
```

6. Ejemplo de Salida Esperada

Sudoku resuelto:

```
5 3 4 6 7 8 9 1 2
6 7 2 1 9 5 3 4 8
1 9 8 3 4 2 5 6 7
8 5 9 7 6 1 4 2 3
4 2 6 8 5 3 7 9 1
7 1 3 9 2 4 8 5 6
9 6 1 5 3 7 2 8 4
2 8 7 4 1 9 6 3 5
3 4 5 2 8 6 1 7 9
```

Métricas:
Variante A (básica):
- Intentos: 18234
- Backtracks: 9012
- Tiempo: 0.25s

Variante B (optimizada):
- Intentos: 4120
- Backtracks: 950
- Tiempo: 0.04s

(Los valores pueden variar, pero debe observarse mejora con la optimización en la mayoría de casos.)

7. Validaciones Obligatorias

- Validar que la entrada sea 9×9.
- Validar que los números estén en rango 0..9.
- Validar que el tablero inicial no viole reglas.
- Si no hay solución, reportar claramente:
    - ERROR: Sudoku sin solución o equivalente.
- Evitar modificaciones de casillas fijas (no vacías).

8. Interfaz (Consola)

Debe incluir un menú que permita:

1. Ingresar tablero
2. Validar tablero inicial
3. Resolver con Variante A (básica)
4. Resolver con Variante B (optimizada)
5. Mostrar métricas comparativas
6. Reiniciar tablero
7. Salir

9. Aspectos de Evaluación Clave

Se evaluará especialmente:

- Correcta implementación del backtracking.
- Correcta verificación de restricciones.

- Implementación real de optimización (poda).
- Métricas comparativas claras.
- Diseño modular (clases, separación de responsabilidades).
- Capacidad de justificar estructuras usadas.

10. Consideraciones en la Defensa

Durante la defensa, los estudiantes deben poder explicar:

- Por qué Sudoku se resuelve bien con backtracking.
- Qué significa “poda” y cómo reduce el espacio de búsqueda.
- Diferencia entre validar recorriendo todo vs validar con estructuras auxiliares.
- Complejidad temporal/espacial y por qué cambia según el tablero.
- Cómo seleccionan la siguiente casilla (y por qué).

Sistema de Rutas (Dijkstra / A*)

1. Descripción General

El proyecto consiste en desarrollar un sistema que calcule la **ruta más corta** entre dos puntos dentro de una red de caminos o conexiones, modelada como un **grafo ponderado**.

- Cada **nodo** representa una ubicación (ciudad, edificio, intersección, estación, etc.).
- Cada **arista** representa un camino entre dos ubicaciones con un **costo** asociado (distancia, tiempo, peaje, consumo, etc.).

El sistema debe permitir ejecutar y comparar algoritmos de búsqueda de caminos:

- **Dijkstra** (para costos no negativos)
- **A*** (búsqueda informada usando heurística)

Además, debe analizar la eficiencia y escalabilidad según el tamaño del grafo.

2. Objetivo del Proyecto

Desarrollar un sistema que:

- Permita construir una red de rutas (grafo) con pesos.
- Permita consultar la ruta más corta entre un origen y un destino.
- Ejecute al menos:
  - Dijkstra
  - A* (si el modelo permite heurística)
- Muestre la ruta y el costo total.
- Compare el desempeño de ambos enfoques (cuando aplique).
- Analice complejidad temporal y espacial, incluyendo métricas reales.

3. Requerimientos Funcionales

El sistema debe permitir:

3.1 Configuración del grafo

- Crear nodos (con identificador único).
- Crear aristas con peso:

  - dirigido o no dirigido (definido por el grupo, pero debe justificarse).
- (Opcional recomendado para A*): asociar coordenadas (x,y) a cada nodo.

3.2 Consultas

- Ingresar nodo origen y nodo destino.
- Calcular la ruta más corta.
- Mostrar:
  - Ruta encontrada (secuencia de nodos)
  - Costo total acumulado

3.3 Reportes

- Mostrar métricas de ejecución por algoritmo:
  - nodos visitados (o expandidos)
  - relajaciones realizadas (Dijkstra)
  - tamaño máximo de frontera
  - tiempo de ejecución
- Comparar resultados entre algoritmos.

4. Requerimientos Técnicos Obligatorios

4.1 Estructuras de Datos

Debe utilizar al menos:

Una estructura lineal

- Lista para representar el camino final
- Cola/cola de prioridad (si se implementa como estructura lineal auxiliar, aunque usualmente es heap)

Una estructura no lineal

- **Grafo** representado con lista de adyacencia (obligatorio)
- **Cola de prioridad** (heap) para seleccionar el siguiente nodo con menor costo (obligatorio para eficiencia)

- **Tabla hash** o estructura de mapa para:
    - distancias
    - nodos previos
    - visitados

No se permite implementar Dijkstra con búsqueda lineal del mínimo en cada iteración sin justificar, ya que degrada el rendimiento. La cola de prioridad debe ser parte del diseño o debe justificarse una estructura equivalente.

4.2 Algoritmos (obligatorio comparar variantes)

Debe implementarse:

Variante A – Dijkstra

- Considera costos no negativos.
- Retorna la ruta de costo mínimo.

Variante B – A*

- Usa una heurística $h(n)$que estime el costo desde un nodo $n$hasta el destino.
- Debe explicar la heurística elegida.
- Ejemplo común: distancia euclidiana o Manhattan usando coordenadas.

Si el grupo no utiliza coordenadas, puede implementar como Variante B una comparación entre:

- Dijkstra con cola de prioridad vs Dijkstra con estructura menos eficiente (y justificar).
- Dijkstra vs BFS en un grafo sin pesos (caso adicional), pero lo ideal es A*.

4.3 Documentación (README)

El README.md debe incluir:

- Complejidad temporal y espacial de:
    - Dijkstra con cola de prioridad: $O(E\log\ V)$
    - A* (depende de heurística, pero discutir reducción de exploración)
- Explicar representación del grafo:

  - lista de adyacencia
  - estructura de nodos/aristas
- Métricas reales por algoritmo:
  - nodos expandidos
  - tamaño máximo de la frontera
  - tiempo
- Comparación en al menos 2 escenarios:
  - Grafo pequeño (pocos nodos)
  - Grafo mediano (más nodos o mayor conectividad)
- Discusión sobre escalabilidad (V y E).

5. Ejemplo de Entrada

Grafo (no dirigido), pesos:

- `A–B: 4`
- `A–C: 2`
- `C–B: 1`
- `B–D: 5`
- `C–D: 8`

Consulta:

- `Origen = A`
- `Destino = D`

(Opcional para A): Coordenadas*

- `A(0,0), B(3,1), C(1,0), D(4,2)`

6. Ejemplo de Salida Esperada

```
Algoritmo: Dijkstra
Ruta: A -> C -> B -> D
Costo total: 8
```

```
Métricas:
Nodos expandidos: 4
Relajaciones: 5
Frontera máxima: 3
Tiempo: 0.002s

Algoritmo: A*
Ruta: A -> C -> B -> D
Costo total: 8

Métricas:
Nodos expandidos: 3
Frontera máxima: 2
Tiempo: 0.001s
Heurística: Euclidiana
```

(Los valores pueden variar, pero A debe expandir igual o menos nodos en escenarios donde la heurística es informativa.)*

7. Validaciones Obligatorias

- Validar que origen y destino existan.
- Manejar:
  - Nodo destino no alcanzable.
  - Grafo sin conexiones.
- Validar que los pesos sean no negativos (para Dijkstra).
- Evitar duplicados en nodos/aristas (o manejarlos correctamente).

8. Interfaz (Consola)

Debe incluir un menú que permita:

1. Agregar nodos
2. Agregar aristas (con peso)
3. Cargar grafo de ejemplo (opcional)
4. Consultar ruta con Dijkstra
5. Consultar ruta con A*

6. Mostrar métricas comparativas
7. Mostrar grafo (lista de adyacencia)
8. Salir

9. Aspectos de Evaluación Clave

Se evaluará especialmente:

- Correcta implementación de lista de adyacencia.
- Uso correcto de cola de prioridad.
- Reconstrucción correcta de la ruta (predecesores).
- Implementación y justificación de heurística en A*.
- Comparación cuantitativa con métricas reales.
- Claridad y calidad del análisis de complejidad.

10. Consideraciones en la Defensa

Durante la defensa, los estudiantes deben poder explicar:

- Diferencia entre BFS, Dijkstra y A*.
- Por qué Dijkstra requiere pesos no negativos.
- Por qué A* es más eficiente cuando la heurística es adecuada.
- Qué significa V y E y cómo afecta el rendimiento.
- Cómo se reconstruye el camino final.
- Cómo y por qué se evita procesar estados repetidos.

Red Social

1. Descripción General

El proyecto consiste en modelar una **red social** donde los usuarios se conectan mediante relaciones de amistad o seguimiento. Este tipo de problema se representa naturalmente como un **grafo**, donde:

- Cada **usuario** es un **nodo**.
- Cada **relación** (amistad/seguimiento) es una **arista**.
    - Puede ser **no dirigida** (amistad) o **dirigida** (seguimiento).

El sistema debe permitir registrar usuarios, crear conexiones y resolver consultas típicas de redes sociales utilizando algoritmos de recorrido/búsqueda en grafos (BFS/DFS), además de generar métricas relevantes.

2. Objetivo del Proyecto

Desarrollar un sistema que:

- Permita registrar usuarios y relaciones.
- Permita realizar consultas sobre el grafo social.
- Implemente algoritmos de recorrido/búsqueda para resolver consultas.
- Compare al menos dos enfoques o variantes (cuando aplique).
- Analice complejidad temporal y espacial y reporte métricas reales.

3. Requerimientos Funcionales

El sistema debe permitir:

3.1 Gestión de usuarios

- Crear usuario (ID único y nombre).
- Listar usuarios existentes.
- Buscar usuario por ID o nombre.

3.2 Gestión de relaciones

- Crear relación entre usuarios:

    - Amistad (no dirigida)
    - Seguimiento (dirigida)
- Eliminar relación (opcional, recomendado).
- Mostrar lista de conexiones de un usuario.

3.3 Consultas obligatorias (mínimo)

Debe incluir al menos estas consultas:

1. **Grado de separación** entre dos usuarios
    - Calcular el mínimo número de saltos entre usuario A y B (BFS).
    - Mostrar el camino (usuarios intermedios).
2. **Recomendación de amigos** (o "a quién seguir")
    - Basado en amigos en común / vecinos de segundo nivel.
    - Debe mostrar un ranking (top N sugerencias).
3. **Usuarios más conectados**
    - Top N por cantidad de conexiones (grado del nodo).
    - (Opcional) centralidad simple basada en BFS (más avanzado).

3.4 Reportes

- Cantidad total de usuarios.
- Cantidad total de relaciones.
- Componentes conexas (opcional, recomendado si la red es no dirigida).
- Densidad aproximada del grafo (opcional).

4. Requerimientos Técnicos Obligatorios

4.1 Estructuras de Datos

Debe utilizar al menos:

Una estructura lineal

Ejemplos aceptables:

- Cola para BFS (obligatorio en grado de separación)
- Lista para mostrar camino
- Lista para ranking y ordenamientos

Una estructura no lineal

Ejemplos aceptables:

- **Grafo** (lista de adyacencia obligatoria)
- **Tabla hash** para:
    - usuarioID → nodo/índice
    - nombre → usuario (opcional)
- (Opcional) conjunto/hash para evitar repetidos en recomendación

No se permite mantener usuarios solo en arreglos sin búsqueda eficiente cuando el sistema crece. La tabla hash debe estar justificada y utilizada.

4.2 Algoritmos (obligatorio comparar variantes)

Debe implementarse:

Variante A – BFS para grado de separación

- Retorna el camino más corto en número de relaciones.
- Reconstrucción del camino usando estructura de predecesores.

Variante B – Recomendación y ranking (dos enfoques)

Comparar al menos dos criterios de recomendación, por ejemplo:

- Enfoque 1: **Amigos en común** (vecinos de vecinos)
- Enfoque 2: **Frecuencia ponderada** (amigos en común + penalización por distancia o por popularidad)

También puede compararse:

- BFS vs DFS en alguna consulta (explicando limitaciones).
- Recomendación por “2 saltos” vs “3 saltos” (más complejo).

4.3 Documentación (README)

El README.md debe incluir:

- Complejidad temporal y espacial de:
  - Agregar usuario
  - Agregar relación
  - Grado de separación (BFS)
  - Recomendación (vecinos de vecinos)
- Métricas reales (medidas con distintos tamaños de red):
  - Tiempo de consulta de separación
  - Cantidad de nodos visitados
  - Tiempo de recomendación
- Comparación entre variantes de recomendación:
  - ¿Cuál genera mejores sugerencias? (justificación)
  - ¿Cuál consume más tiempo/memoria?

5. Ejemplo de Entrada

Usuarios:

- Ana
- Luis
- Marta
- José
- Carla

Relaciones (amistad):

- Ana–Luis
- Luis–Marta
- Marta–José
- Luis–Carla

Consultas:

- Grado de separación: Ana → José
- Recomendación para Ana (Top 3)

6. Ejemplo de Salida Esperada

```
Grado de separación Ana -> José: 3
Camino: Ana -> Luis -> Marta -> José

Recomendaciones para Ana (Top 3):
1) Marta (1 amigo en común: Luis)
2) Carla (1 amigo en común: Luis)
3) José (camino de 3 saltos)

Usuarios más conectados:
1) Luis (3 conexiones)
2) Marta (2 conexiones)
3) Ana (1 conexión)

Métricas:
BFS separación:
- Nodos visitados: 4
- Tiempo: 0.001s

Recomendación:
- Candidatos evaluados: 3
- Tiempo: 0.002s
```

(Los valores pueden variar, pero deben ser coherentes.)

7. Validaciones Obligatorias

- No permitir usuarios duplicados (por ID).
- No permitir relaciones consigo mismo.
- Manejar relaciones duplicadas:
    - Evitar duplicar aristas o controlar con set/hash.
- Manejar consultas con usuarios inexistentes.
- Si no hay conexión entre A y B:

- Reportar “No existe camino” claramente.

8. Interfaz (Consola)

Debe incluir un menú que permita:

1. Crear usuario
2. Crear relación (amistad/seguimiento)
3. Mostrar conexiones de un usuario
4. Calcular grado de separación
5. Recomendar amigos (Top N)
6. Mostrar usuarios más conectados (Top N)
7. Cargar caso de prueba (opcional)
8. Mostrar métricas
9. Salir

9. Aspectos de Evaluación Clave

Se evaluará especialmente:

- Uso correcto de lista de adyacencia.
- Uso real de cola (BFS) para separación.
- Reconstrucción correcta del camino.
- Recomendación basada en estructura de grafo (no “hardcode”).
- Ranking y métricas comparativas.
- Diseño modular (clases Usuario, Grafo, ServicioConsultas, etc.).

10. Consideraciones en la Defensa

Durante la defensa, los estudiantes deben poder explicar:

- Por qué una red social se modela como grafo.
- Diferencia entre grafo dirigido y no dirigido.
- Por qué BFS da el camino mínimo en número de saltos.

- Cómo evitan usuarios repetidos y relaciones duplicadas.
- Complejidad de recomendar amigos en común (y cómo escala).

Compresor Huffman

1. Descripción General

El proyecto consiste en desarrollar un sistema de compresión de texto basado en el **algoritmo de Huffman**, el cual asigna códigos binarios de longitud variable a los caracteres según su frecuencia:

- Caracteres **más frecuentes** → códigos **más cortos**
- Caracteres **menos frecuentes** → códigos **más largos**

El algoritmo construye un **árbol binario** (Árbol de Huffman) utilizando una **cola de prioridad** (min-heap) para combinar los dos nodos de menor frecuencia repetidamente hasta formar un solo árbol.

El sistema debe permitir:

1. Analizar frecuencias
2. Construir el árbol de Huffman
3. Generar la tabla de códigos
4. Codificar (comprimir) un texto
5. Decodificar (descomprimir) para verificar integridad
6. Reportar métricas de compresión y eficiencia

2. Objetivo del Proyecto

Desarrollar un sistema que:

- Permita ingresar un texto (o cargarlo desde consola).
- Calcule frecuencias de caracteres.
- Construya el árbol de Huffman.
- Genere códigos binarios prefijo-libres.
- Comprima el texto y muestre el resultado (o una representación).
- Descomprima el texto y valide que coincida con el original.
- Compare al menos dos enfoques o variantes cuando sea aplicable.

- Analice complejidad temporal y espacial, y reporte métricas reales.

3. Requerimientos Funcionales

El sistema debe permitir:

3.1 Entrada

- Ingresar una cadena de texto (mínimo 20 caracteres para pruebas).
- (Opcional) Permitir espacios y signos de puntuación.
- (Opcional) Permitir modo “archivo” si usted lo permite; si no, solo consola.

3.2 Construcción de Huffman

- Calcular la frecuencia de cada símbolo.
- Construir el árbol de Huffman.
- Generar tabla de códigos por carácter.

3.3 Compresión

- Codificar el texto original en bits (string de 0/1 o estructura bitset).
- Mostrar:
    - tamaño original (en bits o bytes)
    - tamaño comprimido (en bits)
    - porcentaje de compresión

3.4 Descompresión

- Decodificar el texto comprimido.
- Verificar coincidencia exacta con el original:
    - Decodificación correcta
    - Error si no coincide.

3.5 Reportes

- Mostrar tabla de frecuencias.
- Mostrar tabla de códigos Huffman.

- Mostrar métricas comparativas.

4. Requerimientos Técnicos Obligatorios

4.1 Estructuras de Datos

Debe utilizar al menos:

Una estructura lineal

- Lista de caracteres/códigos para imprimir reportes
- Pila (opcional) para recorridos del árbol
- Lista para almacenar el texto comprimido/decodificado

Una estructura no lineal

- **Árbol binario** (Árbol de Huffman) (obligatorio)
- **Cola de prioridad** (min-heap) para construir el árbol (obligatorio)
- **Tabla hash / mapa** para frecuencias y códigos (obligatorio)

No se permite construir el árbol "a mano" o con ordenamientos repetidos sin cola de prioridad, salvo que se justifique formalmente el impacto en eficiencia (no recomendado).

4.2 Algoritmos (obligatorio comparar variantes)

Debe implementarse:

Variante A – Huffman estándar

- Calcular frecuencias
- Construir árbol usando min-heap
- Generar códigos recorriendo el árbol (DFS)

Variante B – Comparación de variante (elegir una)

Se debe comparar Huffman contra **alguna variante o escenario**, por ejemplo:

- **Comparación por distribución**:
    - Texto con caracteres muy repetidos vs texto uniforme
    - Comparar % de compresión
- **Comparación con codificación fija**:

  - ASCII fijo de 8 bits por caracter vs Huffman
- (Opcional) Huffman con "tie-break" determinista para reproducibilidad

Lo importante es que exista comparación cuantitativa: no solo "Huffman funciona", sino "cuándo mejora y cuánto".

4.3 Documentación (README)

El README.md debe incluir:

- Complejidad temporal y espacial de:
  - Conteo de frecuencias: $O(n)$
  - Construcción del árbol: $O(k\log\ k)$donde $k$es el número de símbolos distintos
  - Codificación/decodificación: $O(n)$en longitud del texto (con discusión)
- Métricas reales:
  - Tamaño original vs comprimido
  - % de compresión
  - Tiempo de compresión y descompresión
- Comparación entre escenarios/variantes:
  - Texto repetitivo vs uniforme (o fijo 8-bit vs Huffman)
- Discusión de escalabilidad:
  - Qué ocurre si aumenta el texto o los símbolos distintos.

5. Ejemplo de Entrada

Texto: ABRACADABRA

6. Ejemplo de Salida Esperada

```
Frecuencias:
A: 5
B: 2
R: 2
C: 1
D: 1
```

```
Códigos Huffman (pueden variar):
A: 0
B: 10
R: 110
C: 1110
D: 1111

Texto comprimido:
0 10 110 0 1110 0 1111 0 10 110 0
(Representación en bits concatenados)

Descompresión:
ABRACADABRA
Decodificación correcta

Métricas:
Tamaño original: 11 caracteres = 88 bits (ASCII 8-bit)
Tamaño comprimido: 23 bits
Compresión: 73.86%

Comparación:
Texto uniforme (ej. ABCDEFGHIJ):
Compresión: ~0% (o menor mejora)
```

(Los códigos y bits exactos pueden variar, pero deben ser prefijo-libres y la decodificación debe coincidir con el texto original.)

7. Validaciones Obligatorias

- Manejar texto vacío (reportar error).
- Manejar texto con un solo símbolo (caso especial de Huffman).
- Validar que la tabla de códigos sea prefijo-libre (por construcción).
- Validar que la descompresión reproduzca exactamente el texto original.

8. Interfaz (Consola)

Debe incluir un menú que permita:

1. Ingresar texto
2. Calcular frecuencias

3. Construir árbol Huffman
4. Mostrar códigos
5. Comprimir texto
6. Descomprimir texto
7. Mostrar métricas y comparación
8. Salir

9. Aspectos de Evaluación Clave

Se evaluará especialmente:

- Construcción correcta del árbol (min-heap).
- Generación correcta de códigos (recorrido del árbol).
- Compresión y descompresión correctas.
- Uso adecuado de hash/map para frecuencias y códigos.
- Comparación cuantitativa real.
- Análisis de complejidad y métricas.

10. Consideraciones en la Defensa

Durante la defensa, los estudiantes deben poder explicar:

- Por qué Huffman produce códigos prefijo-libres.
- Por qué usar min-heap reduce el costo de construcción.
- Diferencia entre codificación fija vs variable.
- Cuándo Huffman comprime mejor (distribución sesgada de frecuencias).
- Complejidad de construir el árbol y de codificar/decodificar.

Simulador de Tráfico Vehicular

1. Descripción General

El proyecto consiste en desarrollar un simulador de tráfico vehicular en una red vial, permitiendo representar **intersecciones, rotondas y tramos de carretera** por donde circulan vehículos (carros y motos). El sistema debe simular el movimiento de los vehículos, las colas que se forman por congestión y el impacto de cambios en infraestructura (por ejemplo: ampliar carriles o agregar una vía alternativa).

La red vial debe modelarse como un **grafo dirigido** o **no dirigido** (según el diseño), donde:

- **Nodo:** intersección, rotonda, entrada/salida, semáforo.
- **Arista:** tramo de vía con atributos:
  - capacidad (vehículos que pueden estar en el tramo o pasar por tick)
  - tiempo base de recorrido
  - número de carriles (afecta capacidad)
  - (opcional) velocidad o prioridad

El sistema debe recolectar métricas de congestión y comparar al menos dos escenarios (o estrategias) para analizar el desempeño.

2. Objetivo del Proyecto

Desarrollar un sistema que:

- Permita construir una red vial (grafo) con parámetros.
- Permita generar vehículos (manual o aleatorio con semilla).
- Simule la circulación durante un número de pasos de tiempo (ticks).
- Gestione colas y congestión por tramo/intersección.
- Calcule rutas (mínimo: BFS o Dijkstra; ideal: Dijkstra/A*).
- Compare al menos dos escenarios:
  - Estrategia de ruteo A vs B.
  - Red base vs red mejorada (infraestructura).
- Analice complejidad temporal y espacial con métricas reales.

3. Requerimientos Funcionales

El sistema debe permitir:

3.1 Configuración de la red vial

- Definir nodos (ID único).
- Definir aristas con:
  - capacidad
  - tiempo de recorrido (en ticks)
  - carriles
- Definir tipo de nodo:
  - intersección simple
  - semáforo (opcional)
  - rotonda (opcional)
- (Opcional recomendado) Coordenadas para visualización simple o heurística.

3.2 Generación de vehículos

Cada vehículo debe tener:

- ID
- Tipo: carro o moto
- Origen y destino
- Tiempo de aparición (tick de entrada al sistema)

El sistema debe permitir:

- Ingreso manual de vehículos
- Generación aleatoria con semilla (recomendado)

3.3 Simulación por ticks (obligatoria)

En cada tick, el simulador debe:

- Insertar nuevos vehículos (si su tiempo de aparición coincide).

- Mover vehículos por la red si:
  - existe capacidad en el tramo siguiente
  - su tiempo de recorrido se cumple
- Administrar colas por tramo o por nodo.
- Actualizar métricas.

3.4 Resultados y reportes

Debe mostrar:

- Total de vehículos ingresados
- Total de vehículos entregados (llegaron al destino)
- Tiempo promedio de viaje
- Tiempo promedio de espera en colas
- Tramos más congestionados (Top N)
- (Opcional) mapa de calor textual (resumen de ocupación por tramo)

4. Requerimientos Técnicos Obligatorios

4.1 Estructuras de Datos

Debe utilizar al menos:

Una estructura lineal

- Cola por tramo (vehículos esperando entrar al tramo)
- Cola de eventos (si se implementa por eventos) o listas por tick
- Lista para historial/estadísticas (o para vehículos completados)

Una estructura no lineal

- **Grafo** con lista de adyacencia (obligatorio)
- **Tabla hash** para:
  - IDVehículo → estado actual
  - (opcional) IDNodo/Tramo → estadísticas

- (Opcional) Cola de prioridad para ruteo (Dijkstra/A*)

No se permite una simulación sin colas reales: la congestión debe modelarse con estructura de cola o equivalente.

4.2 Algoritmos (obligatorio comparar variantes)

Debe implementarse y compararse al menos uno de estos pares:

Opción A (recomendada): Comparación de ruteo

- **Estrategia 1 (base):** ruta fija "más corta por distancia/tiempo base" (Dijkstra con pesos estáticos)
- **Estrategia 2 (mejorada):** ruta que considera congestión (peso dinámico)
    - por ejemplo: costo = tiempo_base + factor * cola_actual

Opción B: Comparación de escenarios (infraestructura)

- Escenario 1: red base
- Escenario 2: red mejorada (aplicar UNA mejora):
    - ampliar carriles en un tramo (capacidad aumenta)
    - agregar vía alternativa (puente a desnivel / bypass)
    - simular accidente (reducción temporal de capacidad)

La comparación debe incluir métricas cuantitativas antes/después.

4.3 Documentación (README)

El README.md debe incluir:

- Modelo de la red: qué es nodo/arista y sus atributos.
- Complejidad temporal y espacial aproximada de:
    - actualización por tick
    - ruteo (si aplica Dijkstra/A*)
- Métricas reales:
    - throughput (vehículos que llegan / total)
    - tiempo promedio de viaje

  - tiempo promedio de espera
  - tramos más congestionados

- Comparación entre variantes (ruteo o escenario).
- Discusión de escalabilidad (qué pasa si aumenta N vehículos, V nodos, E aristas).

5. Ejemplo de Entrada

```
Red (grafo dirigido):

Nodos: A, B, C, D

Tramos:
A->B (cap=2, tiempo=2, carriles=1)
B->C (cap=1, tiempo=2, carriles=1)
A->D (cap=1, tiempo=4, carriles=1)
D->C (cap=1, tiempo=2, carriles=1)

Vehículos:

v1 carro  A->C  aparece t=0
v2 moto   A->C  aparece t=1
v3 carro  A->C  aparece t=1

Simulación:

T = 15 ticks
```

6. Ejemplo de Salida Esperada

```
Resultados:
Vehículos ingresados: 3
Vehículos completados: 3

Tiempo promedio de viaje: 6.7 ticks
Tiempo promedio de espera: 2.0 ticks

Tramos más congestionados (Top 2):
1) B->C  (cola máxima = 2)
2) A->B  (cola máxima = 1)

Comparación de escenarios:
Escenario base: promedio viaje = 6.7
```

```
Mejora aplicada: ampliar carriles en B->C (1->2)
Escenario mejorado: promedio viaje = 5.1
```

(Los valores pueden variar, pero deben ser coherentes con capacidad y colas.)

7. Validaciones Obligatorias

- Validar que nodos y tramos existan.
- Validar que capacidad y tiempo sean positivos.
- Validar que origen/destino existan para cada vehículo.
- Manejar vehículo sin ruta disponible:
    - reportar como “no entregado” o “ruta imposible”.
- Si se usa ruteo dinámico, evitar ciclos infinitos (control de visitados).

8. Interfaz (Consola)

Debe incluir un menú que permita:

1. Crear/red de ejemplo (cargar caso)
2. Agregar nodo
3. Agregar tramo (con capacidad/tiempo/carriles)
4. Generar vehículos (manual o aleatorio con semilla)
5. Ejecutar simulación (T ticks)
6. Mostrar estado actual (opcional)
7. Mostrar métricas y reportes (Top N congestión)
8. Comparar escenarios/estrategias
9. Salir

9. Aspectos de Evaluación Clave

Se evaluará especialmente:

- Modelado correcto del grafo y atributos.
- Uso real de colas para congestión.
- Simulación consistente por ticks.

- Ruteo correcto (si aplica) y comparación con métricas.
- Diseño modular (clases para Red, Tramo, Vehículo, Simulador, Métricas).
- Análisis cuantitativo real y bien documentado.

10. Consideraciones en la Defensa

Durante la defensa, los estudiantes deben poder explicar:

- Cómo representan la red vial como grafo.
- Cómo se modela congestión (colas, capacidad).
- Diferencia entre costos estáticos vs dinámicos.
- Por qué una mejora estructural reduce tiempos (si aplica).
- Complejidad por tick y por ruteo.
- Qué parámetros hacen que el sistema “colapse” (muchos vehículos / baja capacidad).

Sistema de Mensajería y Entrega de Paquetes

1. Descripción General

El proyecto consiste en desarrollar un sistema que simule una **plataforma de mensajería y entrega de paquetes** dentro de una ciudad o región. El sistema debe gestionar:

- Paquetes con distinta prioridad (normal/urgente).
- Centros de distribución (sucursales/bodegas).
- Repartidores (mensajeros) con capacidad limitada.
- Rutas entre ubicaciones (red vial) modeladas como **grafo ponderado**.
- Planificación de entregas (asignación de paquetes a repartidores) y simulación de tiempos.

El sistema debe calcular rutas (mínimo costo/tiempo), administrar colas de paquetes pendientes y medir desempeño (tiempos, entregas completas, eficiencia de asignación).

2. Objetivo del Proyecto

Desarrollar un sistema que:

- Permita modelar una red de ubicaciones conectadas (grafo).
- Permita registrar paquetes con origen y destino.
- Permita registrar repartidores y su capacidad (máximo de paquetes o peso).
- Planifique entregas asignando paquetes a repartidores según una estrategia.
- Calcule rutas para cada entrega (Dijkstra o A*).
- Simule la operación durante un período (ticks o eventos).
- Compare al menos dos estrategias de planificación/asignación.
- Analice complejidad temporal y espacial con métricas reales.

3. Requerimientos Funcionales

El sistema debe permitir:

3.1 Configuración de la red logística (grafo)

- Crear ubicaciones (nodos): centros, barrios, clientes, bodegas, etc.

- Crear rutas (aristas) con peso:
    - distancia o tiempo de viaje
    - (opcional) peaje o costo adicional
- Definir si el grafo es dirigido o no dirigido (justificar).

3.2 Registro de paquetes

Cada paquete debe incluir como mínimo:

- ID único
- Origen (nodo)
- Destino (nodo)
- Prioridad: normal / urgente
- Peso (opcional recomendado)
- Tiempo de creación (para métricas)

El sistema debe permitir:

- Registrar paquetes manualmente
- Generación aleatoria con semilla (recomendado)

3.3 Registro de repartidores

Cada repartidor debe incluir:

- ID
- Ubicación actual (nodo)
- Capacidad máxima (paquetes o peso total)
- Estado: disponible / en ruta

3.4 Planificación y asignación (obligatorio)

El sistema debe:

- Tomar paquetes pendientes
- Asignarlos a repartidores según una estrategia

- Calcular ruta para cada entrega

3.5 Simulación

Debe implementarse una simulación con uno de estos enfoques:

- **Por ticks (recomendado):**
  - cada tick avanza tiempo
  - repartidores se mueven por rutas
  - se entregan paquetes al llegar a destino
- **Por eventos (opcional):**
  - cola de prioridad con eventos (salida, llegada, entrega)

3.6 Reportes

Debe mostrar:

- Paquetes entregados vs pendientes
- Tiempo promedio de entrega
- Tiempo promedio de espera antes de salir (en cola)
- Repartidor más eficiente (mayor entregas / menor tiempo)
- (Opcional) zonas con más paquetes / cuellos de botella

4. Requerimientos Técnicos Obligatorios

4.1 Estructuras de Datos

Debe utilizar al menos:

Una estructura lineal

- Cola de paquetes pendientes (FIFO) o lista
- Lista de entregas realizadas (historial)
- Cola de rutas/acciones por repartidor (plan de viaje)

Una estructura no lineal

- **Grafo** con lista de adyacencia (obligatorio)

- **Tabla hash** para:
  - IDPaquete → estado/ubicación
  - IDRepartidor → estado/ubicación
- **Cola de prioridad** para paquetes urgentes o para eventos (obligatorio si el enfoque lo requiere)

No se permite asignación "manual" sin estructuras; la planificación debe ser automática y basada en reglas.

4.2 Algoritmos (obligatorio comparar variantes)

Debe implementarse y compararse al menos:

Variante A – Estrategia de asignación base

Ejemplos:

- FIFO: asignar paquetes en orden de llegada al primer repartidor disponible.
- Urgentes primero: prioridad por tipo, luego FIFO.

Variante B – Estrategia optimizada

Ejemplos (escoger al menos una):

- "Más cercano": asignar el paquete al repartidor cuya ubicación minimiza distancia al origen (o al destino).
- "Menor carga": asignar al repartidor con menos paquetes asignados.
- "Mixta": urgencia + cercanía + carga (con ponderación).

Además, debe usarse al menos un algoritmo de ruta:

- **Dijkstra** (obligatorio)
- **A*** (opcional, si incluyen heurística)

4.3 Documentación (README)

El README.md debe incluir:

- Modelado del grafo y atributos.
- Complejidad temporal y espacial (Big-O) de:

  - Inserción de paquete
  - Asignación de paquetes
  - Cálculo de ruta (Dijkstra/A*)
  - Actualización por tick
- Métricas reales comparativas:
  - % de entregas completadas
  - tiempo promedio de entrega
  - tiempo promedio de espera en cola
  - eficiencia por repartidor
- Comparación clara de estrategias (A vs B).
- Discusión de escalabilidad:
  - qué sucede al aumentar paquetes, repartidores o nodos.

5. Ejemplo de Entrada

```
Red (nodos y rutas con tiempo):

Nodos: Centro, A, B, C
Rutas:
Centro-A (2)
Centro-B (3)
A-C (4)
B-C (2)

Repartidores:

R1 en Centro, capacidad=2
R2 en B, capacidad=1

Paquetes:

P1: Centro -> C urgente (t=0)
P2: A -> C normal (t=1)
P3: B -> A normal (t=2)

Simulación:

T = 10 ticks
```

6. Ejemplo de Salida Esperada

```
Estrategia A (FIFO):
Entregas completadas: 3/3
Tiempo promedio de entrega: 6.0
Tiempo promedio de espera en cola: 1.3
Repartidor más eficiente: R1 (2 entregas)

Estrategia B (Urgente + Más cercano):
Entregas completadas: 3/3
Tiempo promedio de entrega: 5.2
Tiempo promedio de espera en cola: 0.8
Repartidor más eficiente: R2 (1 entrega rápida)

Métricas adicionales:
Rutas calculadas (Dijkstra): 3
Nodos expandidos promedio: 4
```

(Los valores pueden variar, pero la comparación debe tener coherencia y justificación.)

7. Validaciones Obligatorias

- Validar que origen/destino existan.
- Validar que no existan IDs duplicados de paquetes o repartidores.
- Manejar:
  - Paquetes sin ruta (destino inaccesible).
  - Repartidores sin capacidad disponible.
- Si hay prioridades:
  - garantizar que urgentes se atiendan según reglas.
- Evitar asignaciones imposibles (ej. paquete más pesado que la capacidad).

8. Interfaz (Consola)

Debe incluir un menú que permita:

1. Crear/cargar red de ejemplo
2. Agregar nodo
3. Agregar ruta (peso)

4. Agregar repartidor
5. Registrar paquete (normal/urgente)
6. Ejecutar simulación (T ticks)
7. Ejecutar estrategia A
8. Ejecutar estrategia B
9. Mostrar métricas y reportes
10. Salir

9. Aspectos de Evaluación Clave

Se evaluará especialmente:

- Modelado correcto del grafo (lista de adyacencia).
- Uso correcto de colas y prioridades para paquetes.
- Planificación automática de entregas (no manual).
- Cálculo correcto de rutas (Dijkstra mínimo).
- Comparación real entre estrategias con métricas.
- Diseño modular (Paquete, Repartidor, Red, Planificador, Simulador, Métricas).
- Capacidad de defensa técnica.

10. Consideraciones en la Defensa

Durante la defensa, los estudiantes deben poder explicar:

- Cómo modelan el sistema como un problema de grafos + colas.
- Por qué Dijkstra sirve para rutas de costo mínimo.
- Cómo influye la estrategia de asignación en los tiempos de entrega.
- Cómo manejan prioridades y capacidad.
- Complejidad por tick y por cálculo de rutas.
- Qué pasa cuando aumenta la cantidad de paquetes (cuellos de botella).